

Configuration Recognition Via Overlapping Boxes

Jad Bassil

1 Objectives

1. Determine a memory-efficient representation of 3D lattice-based modular robots through a distributed message-passing algorithm. The algorithm must account for holes inside the structure.
2. Understand and implement an algorithm given its description and pseudo-code.
3. Provide a complexity analysis of your algorithm.
4. Validate and evaluate the algorithm on different configuration.
5. Compare this the overlapping boxes algorithm with border tracing.

2 Work to return for evaluation

Read and implement the overlapping boxes algorithm described in 3 in *VisibleSim* with the *Blinky Blocks* modules. Test it on the several configurations and upload the code, a description of your work and results to moodle. You must include:

1. The list of members of the group (2 people max).
2. Your source directory.
3. Your configuration xml files.
4. Screenshots showing the different configurations and found boxes.
5. Challenge: extend the algorithm to work on 3D configurations by expanding boxes along the $\vec{Z}$ axis.
6. Report the number of messages and simulator time for each tested configuration.
7. Compare the overlapping boxes algorithm with the border tracing algorithm in terms of time, memory and communication.
8. Provide a time, memory and communication complexity.

Your report must be written in LaTeX.

3 Algorithm Description

The main idea of the algorithm is to find a set of full boxes that cover the whole configuration. In this section, we describe the distributed algorithm, shown in Algorithms 1 and ?? for finding the boxes on modular robotic systems where modules communicate using message-passing with their directly attached neighbors.

We assume that all modules share the same 3D coordinate system and each module stores its coordinates in its memory. The algorithm is not affected by the orientation of the coordinate axis.

However, for simplicity, we use the orientation of the axis and the direction notation as shown in Fig. 1. A Box B is defined by two vectors $C_{min}(x_{min}, y_{min}, z_{min})$ and $C_{max}(x_{max}, y_{max}, z_{max})$.

$$X(x, y, z) \in B \Leftrightarrow \begin{cases} x_{min} \leq x \leq x_{max} \\ y_{min} \leq y \leq y_{max} \\ z_{min} \leq z \leq z_{max} \end{cases} \quad (1)$$

Let M denote the set of modules. A straight sequence of connected modules in one direction is referred to as a line of modules. The initial step of the algorithm involves determining the values d_m for every module $m \in M$. The value d_m represents the rank of the module in the line (from 1 to n), moving 'backward' along the $\vec{Y}$ axis starting from module m , until an empty position is reached.

To compute d_m , the algorithm proceeds as follows (see example Fig. 1a) :

1. Initially, modules without an attached neighbor in the backward direction must set d to 1.

Algorithm 1: Distributed shape recognition

```

input:  $(x, y, z)$ , neighbors
1 Initialization:
2 | if  $empty((x, y + 1, z))$  then
3 | | send SET_D_MSG(1) to  $neighbor((x, y - 1, z))$ 
4 Msg Handler SET_D_MSG( $d_{sent}$ ):
5 |  $d \leftarrow d_{sent} + 1$ 
6 | if  $\neg empty((x, y - 1, z))$  then
7 | | send SET_D_MSG( $d$ ) to  $neighbor((x, y - 1, z))$ 
8 | else
9 | | if  $isRmin$  then
10 | | | if  $\neg empty((x + 1, y, z))$  then
11 | | | | send FIND_W_MSG( $id, d$ ) to  $neighbor((x + 1, y, z))$ 
12 | | | else
13 | | | |  $w = 1$ 
14 | | | | Notify front line of  $w$ 
15 | | | |  $myBox = (\{x, y, z\}, \{x + w - 1, y + d - 1, z + h - 1\})$ 
16 Msg Handler FIND_W_MSG( $id, d_{sent}$ ):
17 | if  $d < d_{sent}$  then
18 | | send SET_W_MSG( $id, d_{sent}, 0$ ) to  $neighbor((x, y - 1, z))$ 
19 | else
20 | | if  $empty((x + 1, y, z))$  then
21 | | |  $w \leftarrow 1$ 
22 | | | Notify back line of  $w$ 
23 | | | send SET_W_MSG( $id, d_{sent}, w$ ) to  $neighbor((x, y - 1, z))$ 
24 | | else
25 | | | send FIND_W_MSG( $id, d_{sent}$ ) to  $neighbor((x + 1, y, z))$ 
26 Msg Handler SET_W_MSG( $id, d_{sent}, w_{sent}$ ):
27 | if  $d_{sent} \geq d$  then
28 | |  $w \leftarrow w_{sent} + 1$ 
29 | | Notify back line of  $w$ 
30 | if  $myid = id$  then
31 | |  $myBox = (\{x, y, z\}, \{x + w - 1, y + d - 1, z\})$ 
32 | else
33 | | send SET_W_MSG( $id, d_{sent}, w_{sent} + 1$ ) to  $neighbor((x, y - 1, z))$ 

```

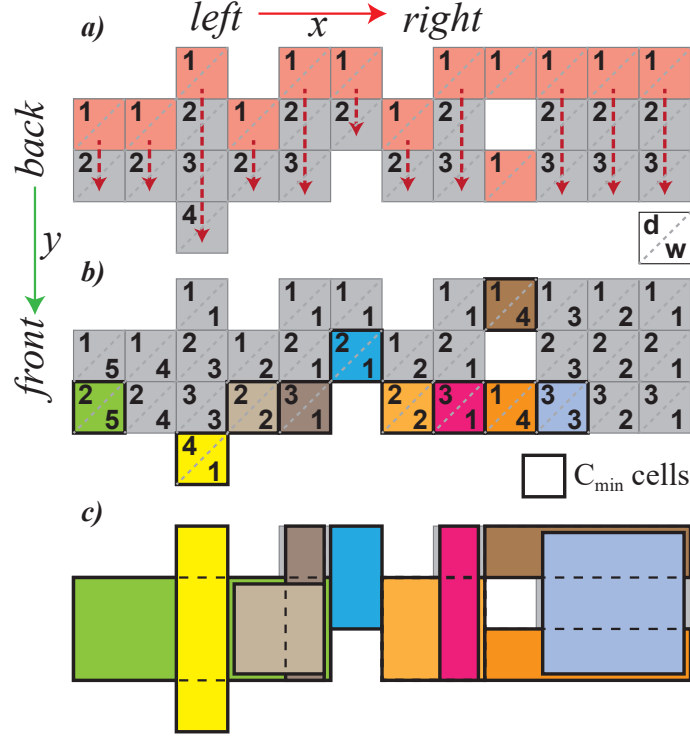


Figure 1: Distributed computation of the boxes on 2D. a) Computation of vertical distance d . b) Election of C_{min} cells and computation of w . c) Boxes covering the plane using C_{min} cells colors.

2. Then, they send a message, denoted as $SET_D_MSG(d)$, to their front neighbor. Upon receiving the message, the module sets its d value as the received value plus one and, subsequently, forwards the message to its front neighbor.
3. This process continues until an empty position is encountered in the front direction (cf. Algorithm 1 lines 1-7).

Once d_m is set, module m can determine locally if it is R_{min} . We denote as d_{left} the d value of the module on the left of the module m . Then, a module is R_{min} if it verifies the following condition:

$$empty(front) \wedge (d \neq d_{left} \vee \neg empty(left + front)) \quad (2)$$

Next, we define w_m for each module m in the set M as the maximum number of connected vertical lines with the same or higher height toward the right (cf. Fig.1b).

We can express w_m by the following rule:

$$w_m = \max_{n \text{ connected}} (y_m = y_n \wedge (d_i \geq d_m \forall i \in [m, n])) \quad (3)$$

Once the value of w_m is determined, we can refer to the values of d and w of the module at the bottom of module m as d_{bottom} and w_{bottom} , respectively. We also use $R_{min}(bottom)$ to indicate whether the bottom module is at an R_{min} position. Then a module m is at a C_{min} position if it verifies:

$$R_{min} \wedge \neg(R_{min}(bottom) \wedge d = d_{bottom} \wedge w = w_{bottom}) \quad (4)$$

To determine w_m for all modules m in the set M , the following distributed process is employed: each module located in a position R_{min} sends a message called $FIND_W_MSG(id, d_{sent})$ to its right neighbor (cf. Algorithm 1 lines 10-11). As the message is forwarded, if the message reaches a module n with $d_n < d_{sent}$, the module n responds to the sender on its left with a message

$\text{SET_W_MSG}(id, d_{sent}, w_{sent} = 0)$. Otherwise, when it reaches a module n that lacks a right neighbor, it assigns w_n the value of 1 and responds to the sender on the left using a message denoted as $\text{SET_W_MSG}((id, d_{sent}, w_{sent} = 1)$. Upon receiving this message, module r sets its w_r value as the received value plus one if and only if $d_{sent} \geq d_r$ to ensure that the line at the back of module r can accommodate d_{sent} modules (cf. Algorithm 1 lines 16-33).

From each R_{min} module, we use local d_m and w_m values to define a rectangle $(R_{min}, R_{max}) = (\{x_m, y_m\}, \{x_m + w_m - 1, y_m + d_m - 1\})$. This approach enables the creation of a vertical decomposition consisting of overlapping rectangles on each 2D layer along the $\vec{Z}$ axis.

The algorithm operates asynchronously. Therefore, a module can receive a FIND_W_MSG before its d value is defined. To solve this, if a module receives a message and the values required for its handling are not yet defined, the module stores the received message in its memory and handles it once the values are set.

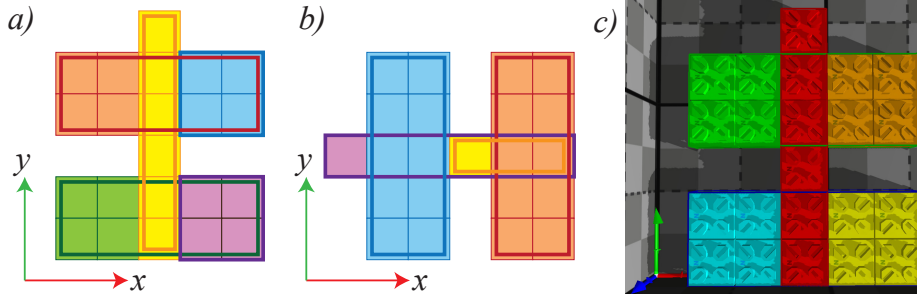


Figure 2: Three basic example: a) A basic shape defined by 5 overlapping boxes. b) The same shape but rotated producing 4 boxes only. c) The same shape computed on *VisibleSim*.

Fig. 2 shows an example of overlapping boxes. It can be seen that the resultant boxes may differ according to the orientation of the coordinate axis. However, in both cases a) and b) the resultant boxes cover all the configuration. Moreover, having an overlapping box results in boxes that are completely inside a bigger one, such as the blue and purple in Fig. 2 (a) and the yellow in Fig. 2 (b). When multiple boxes must be stored, they can be aggregated by neglecting the boxes that are inside another one.